CS 305: Computer Networks

Fall 2025

Lecture 3： Application Layer

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 2: outline

2.1 principles of network applications

2.2 Web and HTTP

2.3 electronic mail

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P applications

2.6 video streaming and content distribution networks

2.7 socket programming with UDP and TCP

Chapter 2: application layer

Our goals:

- Conceptual, implementation aspects of network application protocols
  - client-server architecture
  - peer-to-peer architecture
  - transport-layer service models
- Learn about protocols by examining popular application-level protocols
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
- Creating network applications
  - socket API

Some network apps

- e-mail
- web
- text messaging
- remote login
- P2P file sharing
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- voice over IP (e.g., Skype)
- real-time video conferencing
- social networking
- search
- …
- …

Creating a network app

Write programs that:

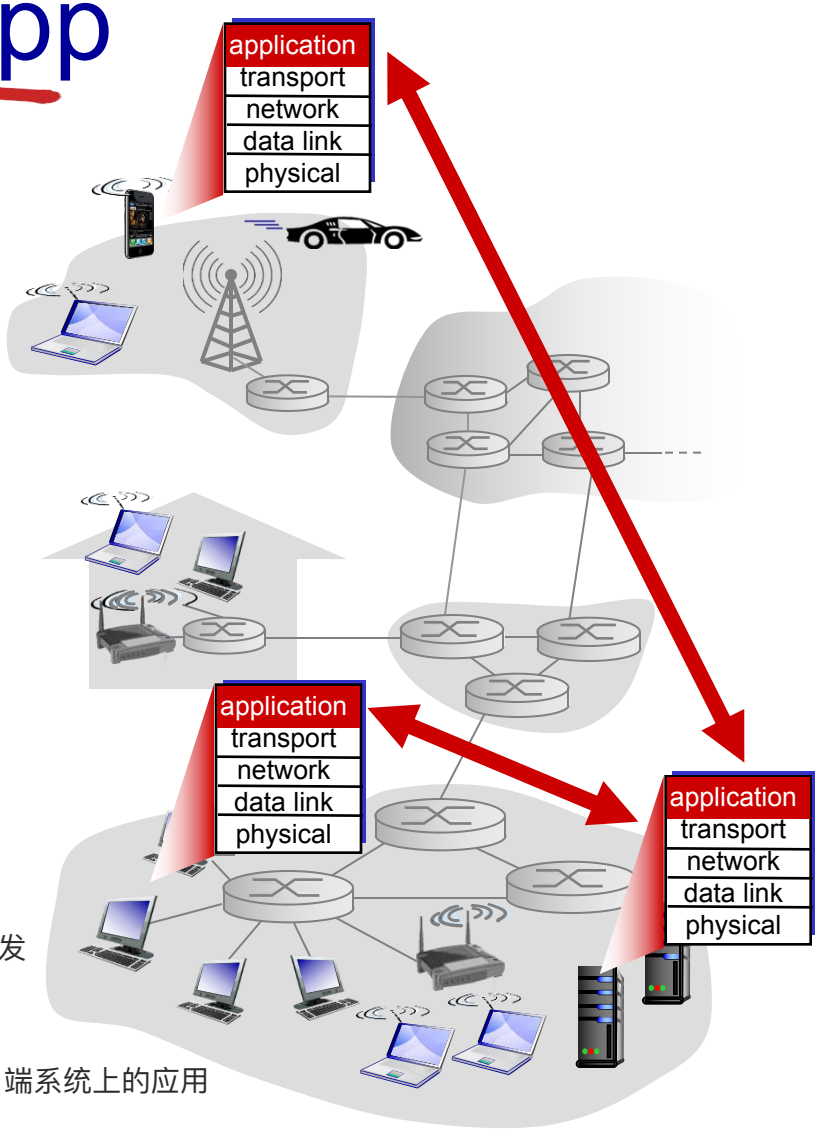
- run on (different) *end systems*
- communicate over network
- e.g., web server software （at server's host） communicates with browser software (at user's host)

No need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation

路由器、交换机等 network–core devices 不运行用户应用，只负责转发数据

所以开发者只需要关心 端系统上的应用



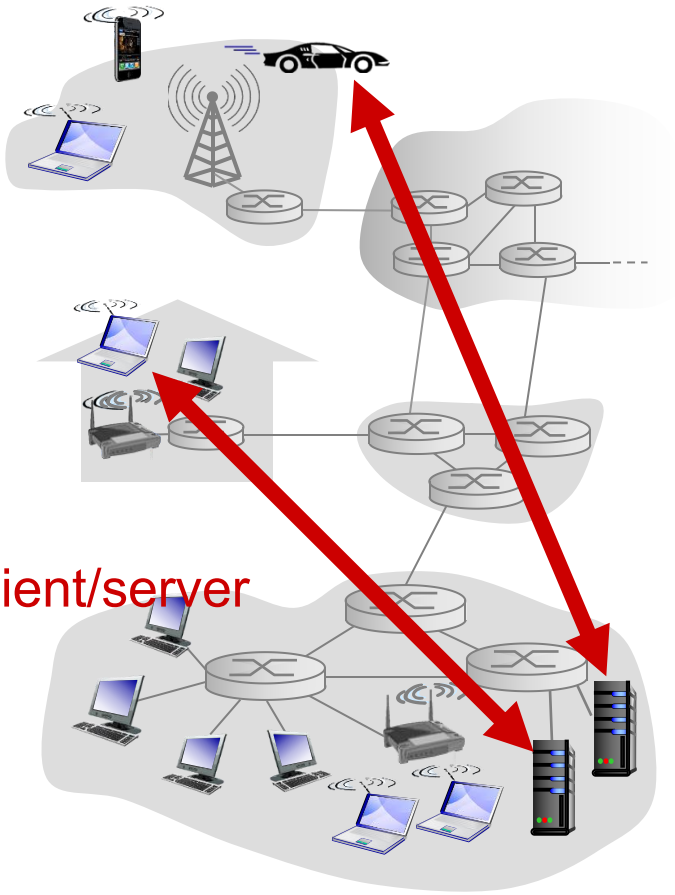
Application architectures

Possible structure of applications:

- client-server
- peer-to-peer (P2P)

没有绝对的“客户端”和“服务器”，节点之间是 对等的

Client-server architecture



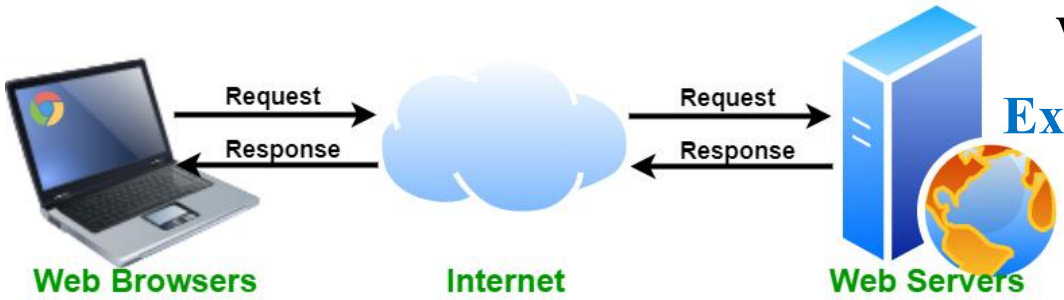
Server:

- always-on host
- Permanent (fixed, well-konwn) IP address 方便客户端找到
- data centers for scaling

Clients:

- communicate with server
- may be intermittently (间接性) connected
- may have dynamic IP addresses
- do not communicate directly with each other 客户端之间 不会直接通信，而是都通过服务器中转

Examples: Web and E-mail

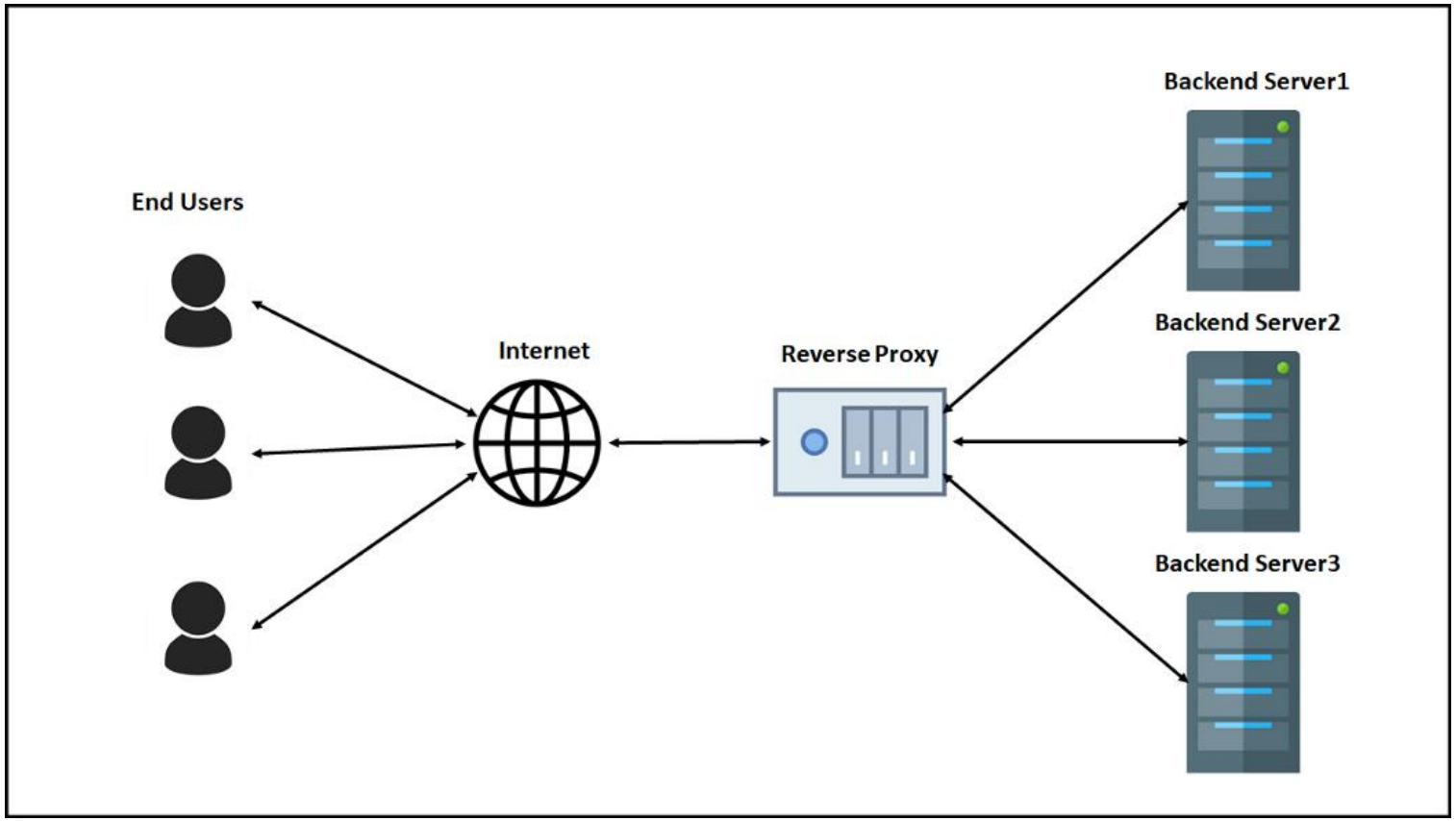


Client-server architecture

Data centers for scaling: a large number of hosts to create a powerful virtual server; distributed around the world

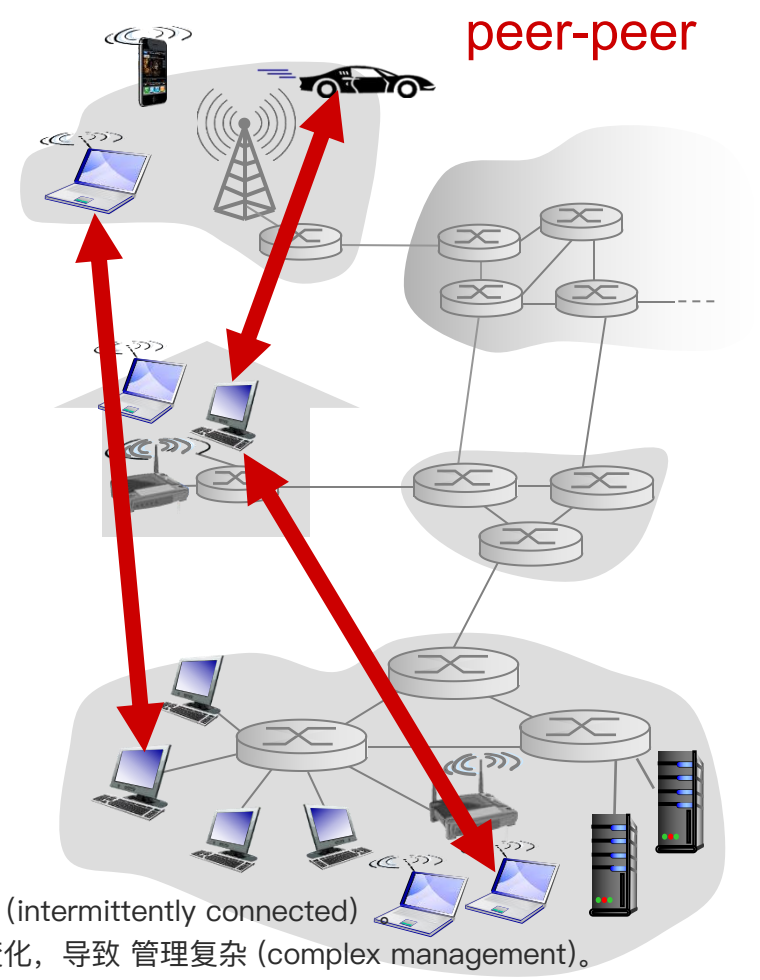
服务器可能需要同时服务成千上万甚至上百万用户，一个单机服务器肯定不够。
于是，很多服务器（主机）被放在 数据中心 里，组成一个强大的虚拟服务器。
这些数据中心 分布在全球（比如图中的 Microsoft Azure），这样可以：
负载均衡（不同地区的用户访问最近的数据中心，减少延迟）。
高可用性（某个数据中心宕机，用户会被分配到另一个）。
扩展性（轻松增加计算能力和存储）。

Client-server architecture



P2P Architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- **Example**: P2P file sharing



Peers are intermittently connected and change IP addresses

- complex management

节点不一定一直在线（intermittently connected）
节点的 IP 地址经常变化，导致 管理复杂 (complex management)。

Hybrid architectures: client-server + P2P

Hybrid Architecture

Zoom

Cloud-Based and Peer-to-Peer Meetings

一开始通过服务器（云端）建立连接（handshake）。
之后的视频数据直接 点对点传输 (P2P)，不再经过服务器。
好处：降低延迟、减轻服务器压力，同时利用服务器来解决“发现对方”的问题

Cloud-hosted solutions manage a handshake between your client and our servers. This initiates your connection to a meeting. For the rest of the meeting, you're streaming your video feed to a server that forwards all of that data to the rest of the participants. Peer-to-peer solutions work by initiating a handshake in a similar manner. Some of them just notify the client on "the other end" that a call is being initiated from such-and-such IP address, and then lets them sort it out between each other. For the rest of the meeting, you're streaming your video feed directly to the other person, and that person will do the same to you. There's no server in-between. The dichotomy that separates the two methods of communication is simply who you're communicating with. With cloud, you're sending your video feed to our server for distribution. With peer-to-peer, you're sending your video feed to the person you're talking to and that's that. Now that we've gotten our point across, let's discuss why these combined solutions work well for you:

How exchange msg?

How do end systems communicate with each other?

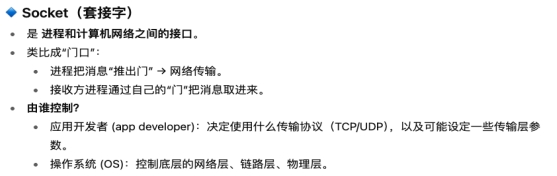
- Who send/recv msg to/from network? Processes (进程)
- Where does process send/recv msg to/from? Socket (套接字)

进程通过它把消息送到网络，或者从网络接收消息

- Processes within same host communicate using inter-process communication (defined by OS)

进程间通信（IPC, inter-process communication），由操作系统提供，比如管道、消息队列、共享内存等

- processes in different hosts communicate by exchanging messages across the computer network



- Socket（套接字）
  - 是 进程和计算机网络之间的接口。
  - 类比成"门口"：
    - 进程把消息"推出门" → 网络传输。
    - 接收方进程通过自己的"门"把消息取进来。
  - 由谁控制?
    - 应用开发者 (app developer)：决定使用什么传输协议（TCP/UDP），以及可能设定一些传输层参数。
    - 操作系统 (OS)：控制底层的网络层、链路层、物理层。

client process: process that initiates communication

server process: process that waits to be contacted

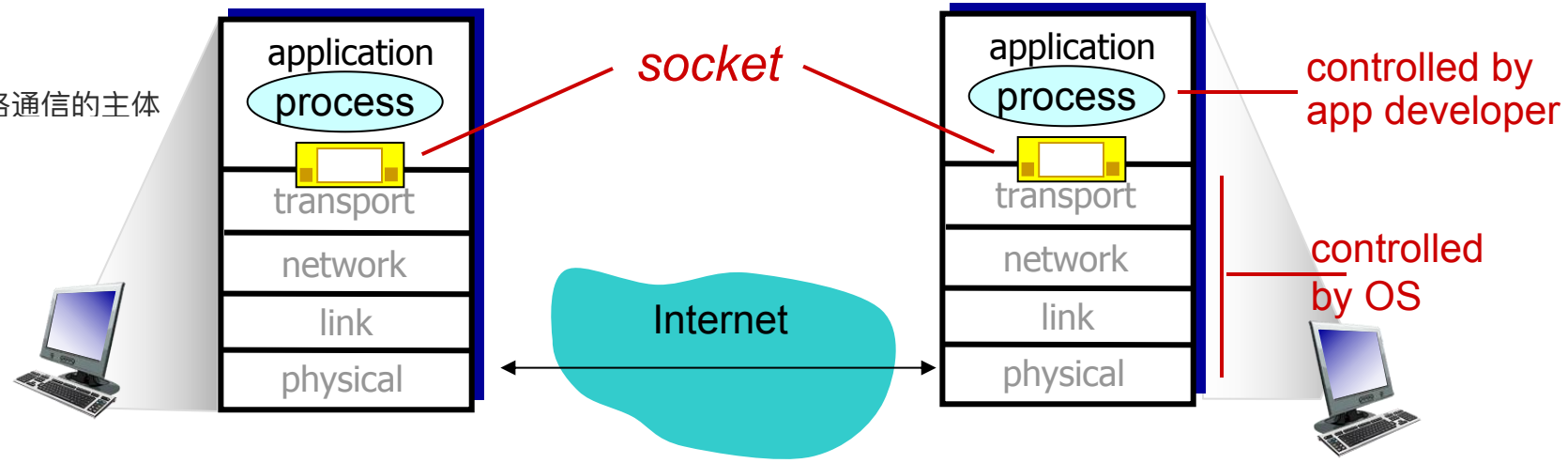
- Client-server architecture
- P2P architectures have client processes & server processes

Interface between Process and Computer Networks: Sockets

- Process sends/receives messages to/from the network through socket
- Socket is analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process

进程收/发网络消息都要通过 socket，就像一扇门：进程把消息“推”到门外 → 网络会负责把它送出去。接收方的进程也通过 socket 收到消息。

进程 是网络通信的主体



The control that the application developer has on the transport-layer side is

- The choice of transport protocol, e.g., UDP, TCP
- Perhaps, the ability to fix a few transport-layer parameters

Addressing Process: IP and Port number

To receive messages, process must have *identifier*

- The address of the host: IP address
- An identifier that specifies the receiving process: port numbers

标识主机上运行的具体进程

Host device has unique 32-bit IP address

Q: Does IP address of host on which process runs suffice for identifying the process?

- *A:* no, *many* processes can be running on same host

Port numbers:

HTTP server: 80

mail server: 25

Example: send HTTP message to gaia.cs.umass.edu web server:

- IP address: 128.119.245.12
- port number: 80

为什么仅有 IP 地址不够?
一台主机可以同时运行很多进程（比如浏览器、邮箱客户端、下载工具）。
仅有 IP 地址只能找到这台主机，却不能区分具体要联系哪个进程。
所以需要 端口号 (port number) 来区分不同进程。

How to choose transport service?

When you develop an application, you must choose one of the available transport-layer protocols (e.g., UDP, TCP):

TCP：提供可靠传输（保证不丢、不乱序），适合文件传输、网页访问。
UDP：不保证可靠，速度快，适合实时应用（语音、视频、游戏）

Reliable data transfer

- delivered correctly, completely, in proper order
- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

Throughput

带宽敏感型应用（Bandwidth-sensitive）：
需要一定速率才能“有效”工作，例如流媒体

- Bandwidth-sensitive applications: require minimum amount of throughput to be “effective”, *r* bits/sec
  - E.g., multimedia

弹性应用（Elastic）：
能利用任何带宽，不要求固定速率，例如邮件、文件下载

- Elastic applications: make use of whatever throughput they get
  - E.g., E-mail, file transfer, web transfer

How to choose transport service?

When you develop an application, you must choose one of the available transport-layer protocols:

Timing

- Real-time applications: some apps require low delay to be "effective"
  - E.g., Internet telephony, interactive games

Security

- encryption, data integrity, …

Transport service requirements: common apps

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
interactive games	loss-tolerant	few kbps up	yes, 100's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

When you create a new network application for the Internet, one of the first decisions you have to make is whether to use UDP or TCP

TCP service:

- *connection-oriented:* setup required between client and server processes
  - TCP connection; full-duplex
- *reliable transport* between sending and receiving process
  - Without error; in proper order; no duplicate bytes
- *congestion control:* throttle sender when network overloaded
- *does not provide:* timing, minimum throughput guarantee, security

全双工通信（两边可以同时发消息）

如果网络太拥挤，TCP 会自动让发送方“慢下来”，避免加重拥堵

UDP service:

- *connectionless*
- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, congestion control, timing, throughput guarantee, security, or connection setup

Q: Why is there a UDP?

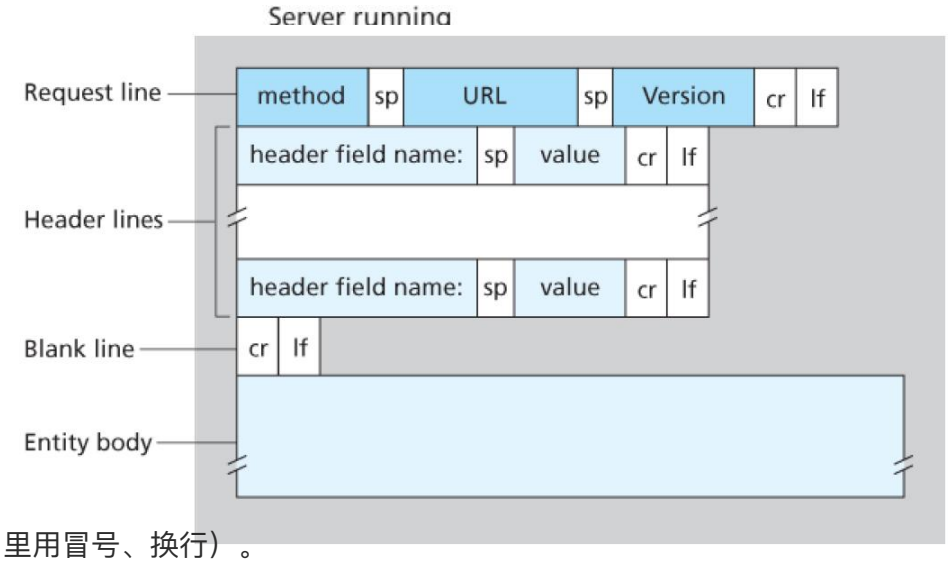
UDP is commonly used in **time-sensitive communications** where occasionally dropping packets is better than waiting.

Internet apps: application, transport protocols

application	application layer protocol	underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

App-layer protocol defines

- types of messages exchanged,
  - e.g., request, response
- message syntax (语法):
  - what fields in messages & how fields are delineated
- message semantics （语义）
  - meaning of information in fields
- rules for when and how processes send & respond to messages



消息里面有哪些字段（fields），
字段之间怎么分隔（比如 HTTP 里用冒号、换行）。

各个字段的意义是什么

open protocols:

- defined in RFCs
- allows for interoperability
- e.g., HTTP, SMTP

Proprietary（专有的） protocols:

- e.g., Skype

Application-Layer Protocols

An application-layer protocol is one piece of a network application.

For example, the Web is a client-server application that allows users to obtain documents from Web servers on demand.

- a standard for document formats (that is, HTML),
- Web browsers (for example, Firefox and Microsoft Internet Explorer)
- Web servers (for example, Apache andMicrosoft servers)
- an application-layer protocol

Chapter 2: outline

2.1 principles of network applications

2.2 Web and HTTP

2.3 electronic mail

- SMTP, POP3, IMAP

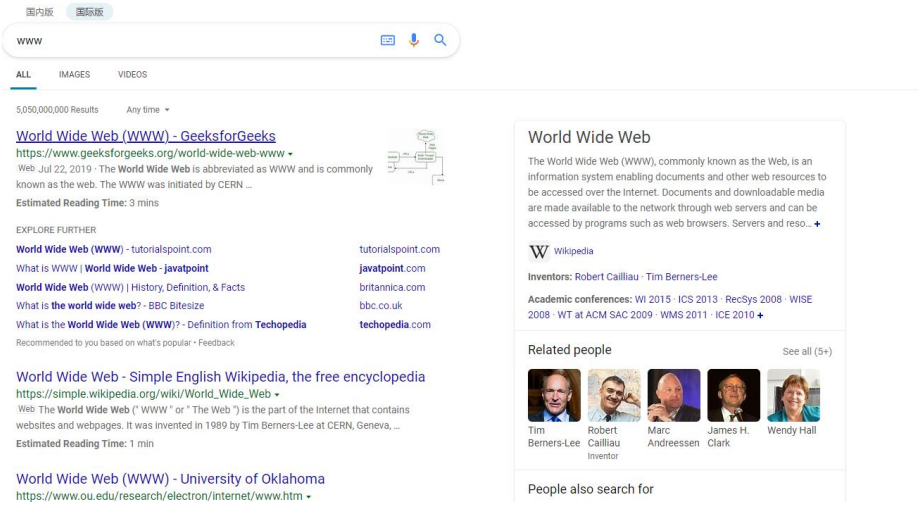
2.4 DNS

2.5 P2P applications

2.6 video streaming and content distribution networks

2.7 socket programming with UDP and TCP

Web



Searching engine

Web-based email

Video streaming platforms

Social networks

Web

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>Example</title>
  </head>
  <body>
    <p>See info about <a href="">dogs</a> or
    <a href="">cats</a></p>
  </body>
</html>
```

- *web page* consists of *objects*
  - object can be HTML file, JPEG image, Java applet （小程序）, audio file,…
- web page consists of *base HTML-file* which includes *several referenced objects*
  - E.g., a HTML text and five JPEG images
- each object is addressable by a *URL,* e.g.,

每个网页有一个 基础 HTML 文件，里面会包含其他对象的引用

每个对象通过 URL（统一资源定位符）来定位

`www.sustc.edu.cn/resources/cn/image/p27.png`

host name　　path name

HTML: hypertext markup language

HTTP: hypertext transfer protocol

HTML (HyperText Markup Language)：网页内容的语言。

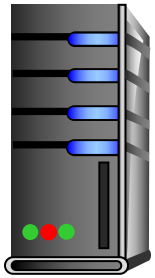
HTTP (HyperText Transfer Protocol)：网页传输的协议。

HTTP and Web

HTTP 是 Web 的核心协议，它定义了客户端和服务器之间如何通信

Web browser/client

HTTP messages



Web server

存放网页内容

HTTP (hypertext transfer protocol) defines

- how Web clients request Web pages from Web servers and
- how servers transfer Web pages to clients

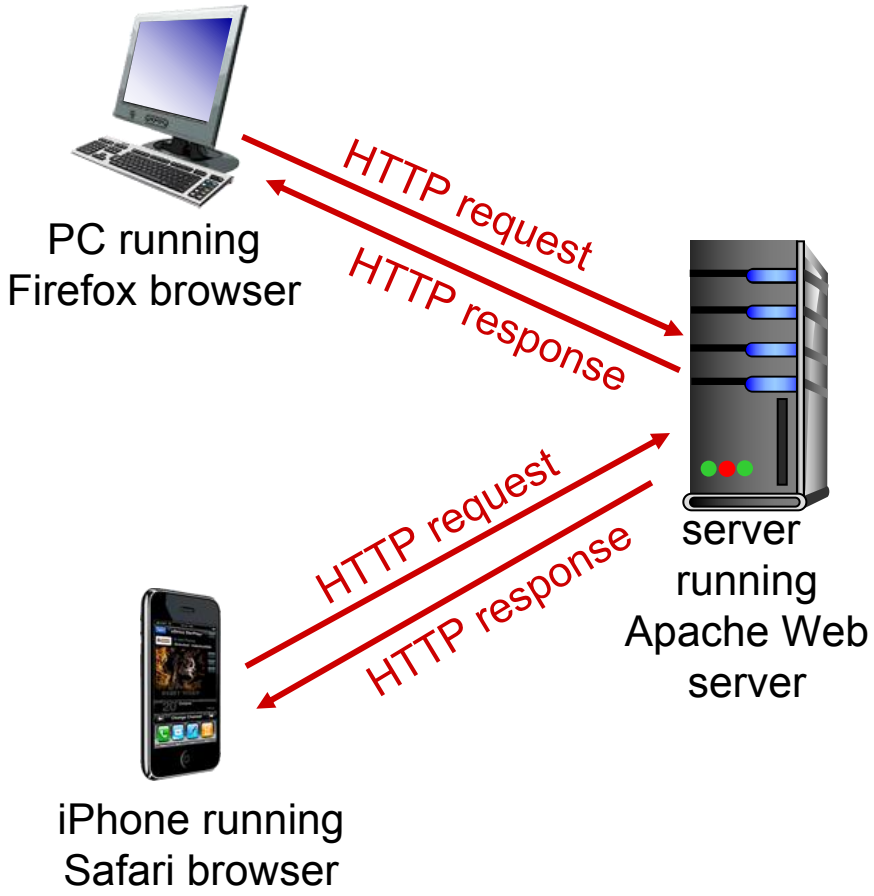
规定了 客户端如何请求 网页内容。
规定了 服务器如何传输 网页内容。

HTTP overview

HTTP: Web's application layer protocol

client/server mode

- *client:* browser that requests, receives, (using HTTP protocol) and "displays" Web objects
- *server:* Web server sends (using HTTP protocol) objects in response to requests

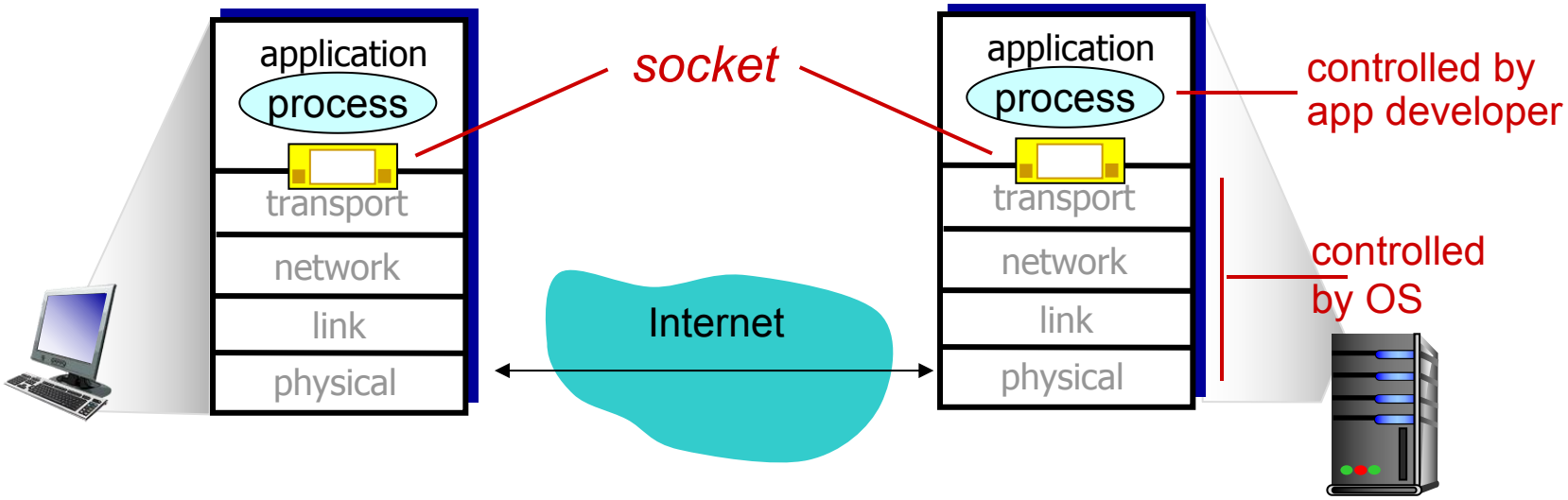


HTTP overview (contir

1. **HTTP 基于 TCP 运行**
   - 客户端通过 **TCP 连接**（端口 80）与服务器通信。
   - 步骤：
     1. 客户端发起 TCP 连接（创建 socket）。
     2. 服务器接受连接。
     3. 双方通过这个连接交换 HTTP 报文（请求/响应）。
     4. 通信完成后，TCP 连接关闭。
2. **HTTP 不需要关心底层细节**
   - HTTP 只管“传送网页内容”。
   - TCP 保证数据不会丢失、不会乱序。
   - 所以 HTTP 不用担心“重传、丢包”等问题。

Uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed



HTTP need not worry about lost data or the details of how TCP recovers from loss or reordering of data.

HTTP overview (continued)

HTTP is "stateless"

- Server maintains no information about past client requests
- If a client asks for the same object twice, the server resends the object.

如果客户端重复请求同一个对象，服务器会再次发送，而不会“记得你上次要过”。

缺点：不能记住历史
如果需要维护用户状态（比如购物车、登录信息），必须通过额外机制（如 cookies、sessions）来实现。

对比有状态协议：
有状态协议需要保存“过去的交互记录”。
如果客户端/服务器崩溃，状态可能不一致，需要处理复杂的恢复

aside

protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

每个请求/响应要不要开一个新的 TCP 连接?

还是说多个请求/响应可以复用同一个 TCP 连接?

Should each request/response pair be sent over a *separate* TCP connection, or should all of the requests and their corresponding responses be sent over the *same* TCP connection?

non-persistent HTTP

- at most one object sent over TCP connection
  - connection then closed
- downloading multiple objects required multiple connections

persistent HTTP

- multiple objects can be sent over single TCP connection between client and server
- default mode

Non-persistent HTTP（非持久连接）Persistent HTTP（持久连接）

多个对象可以复用同一个 TCP 连接。

客户端和服务器之间保持连接，不必反复建立/关闭。

优点：效率更高，HTTP/1.1 默认采用这种模式。

- 每次请求/响应都单独建立一个 TCP 连接。
- 一个对象（比如一个 HTML 文件）传输完成后，TCP 连接就关闭。
- 下载多个对象需要建立多个 TCP 连接。
- 缺点：效率低（频繁建立/断开 TCP 连接）。

Persistent HTTP（持久连接）

- 多个对象可以复用同一个 TCP 连接。
- 客户端和服务器之间保持连接，不必反复建立/关闭。
- 优点：效率更高，HTTP/1.1 默认采用这种模式。

Non–persistent HTTP

suppose user enters URL:

`www.someSchool.edu/someDepartment/home.index`

(contains text, references to 10 jpeg images)

1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80

1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Non-persistent HTTP (cont.)

4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

(As the browser receives the Web page, it displays the page to the user. HTTP has nothing to do with how a Web page is interpreted by a client.)

当浏览器接收到网页时，它会把页面展示给用户。但 HTTP 协议并不管浏览器是如何解析和渲染网页的。

Non-persistent HTTP: each TCP connection transports exactly one request message and one response message.

Users can configure modern browsers to control the degree of parallelism, i.e., multiple TCP in parallel.

Non-persistent HTTP: response time

RTT (round-trip time): time for a small packet to travel from client to server and back

- Propagation, queuing, processing

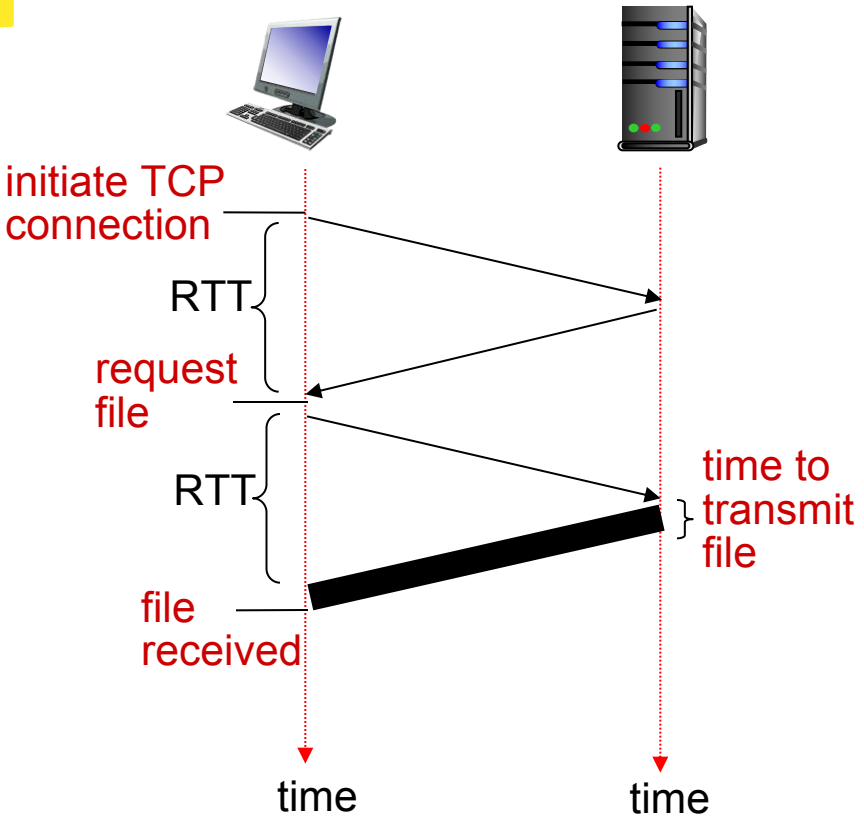
每次请求/响应都要建立一个新的 TCP 连接，用完就关闭

When a user clicks on a hyperlink:

HTTP response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time
- non-persistent HTTP response time =

  2RTT+ file transmission time



从服务器把文件内容完整传输到客户端所需要的时间：文件大小/有效带宽

Persistent HTTP

non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead (TCP buffer, variables) for *each* TCP connection

操作系统开销大：要不断分配 TCP 缓冲区、变量等

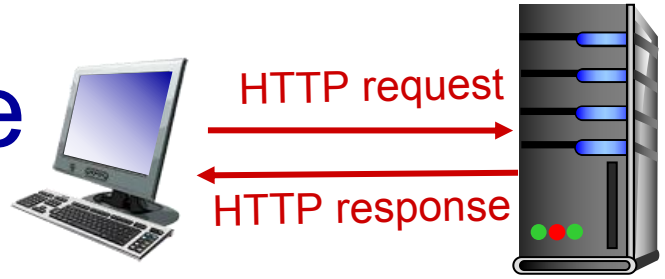
- browsers often open parallel TCP connections to fetch referenced objects

一般会开多个并行 TCP 连接、一次性并行下载多个对象

persistent HTTP:

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- server closes a connection when it isn't used for a certain time
- as little as one RTT for all the referenced objects

HTTP request message



- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)

carriage return (回车) character

line-feed （换行） character

request line (GET, POST, HEAD commands)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

header lines

Browser type: server can send different version of the same object

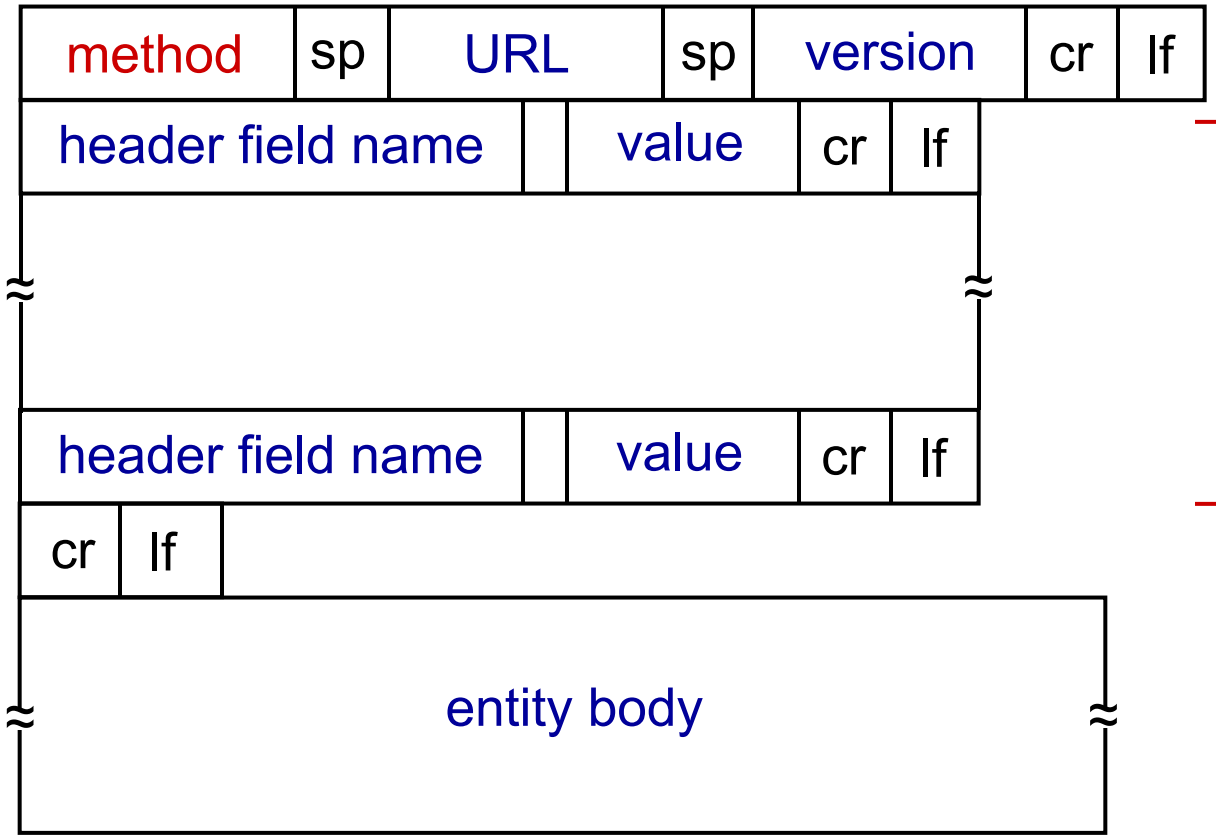
carriage return, line feed at start of line indicates end of header lines

Connection: close

Method filed, URL field, HTTP version field

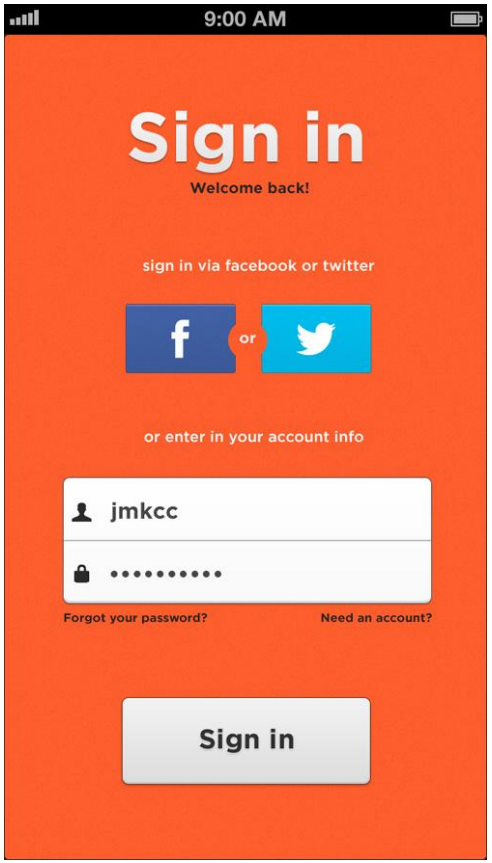
HTTP request message: general format

GET, POST, HEAD, PUT, DELETE



For example, the entity body is used with the POST method (e.g., search words to a search engine).

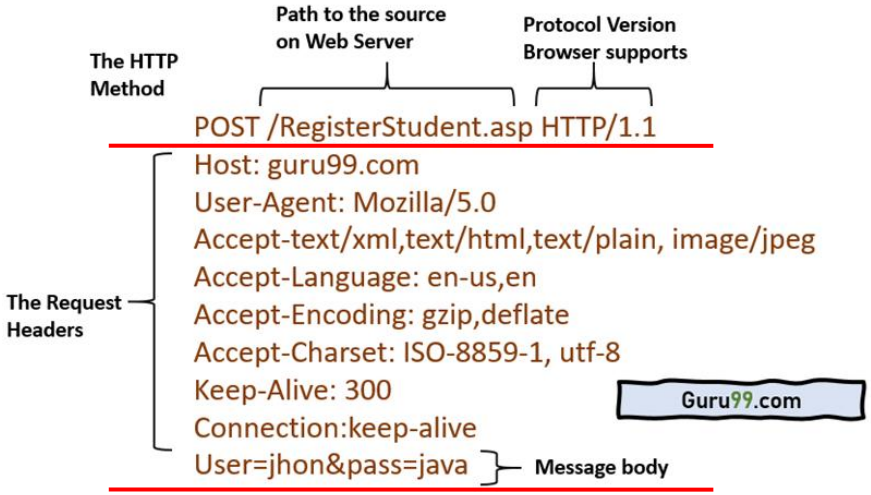
HTTP request message: general format



Uploading form input

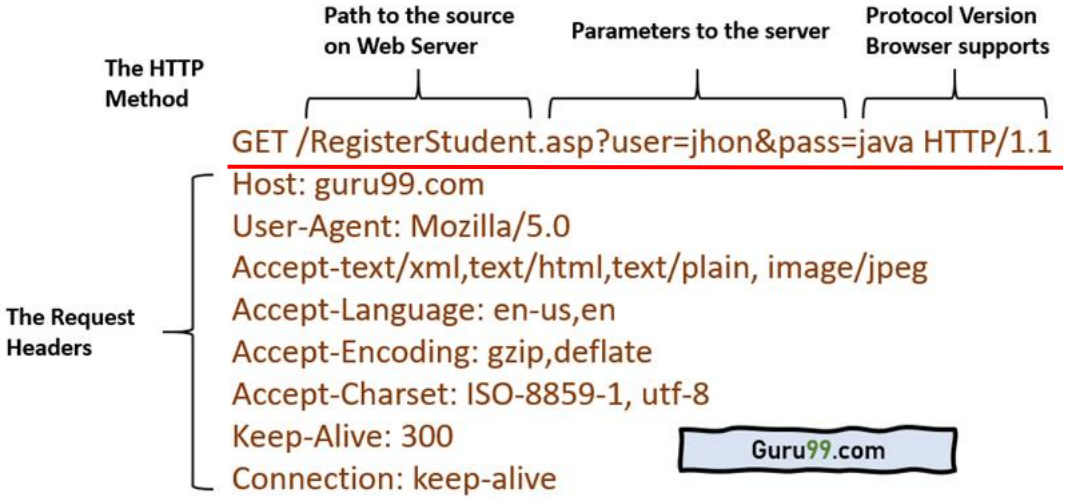
POST method:

- web page often includes form input
- input is uploaded to server in entity bodys



URL method:

- uses GET method
- input data is included in URL field of request line



Method types

HTTP/1.0:

- GET, POST
- HEAD
  - Similar to the *GET* method
  - Server responds with an HTTP message but it leaves out the requested object
  - Used for debugging

HTTP/1.1:

- GET, POST, HEAD
- PUT
  - uploads file in entity body to path specified in URL field
- DELETE
  - deletes file specified in the URL field

HTTP response message

HTTP request

HTTP response

status line
(protocol version
status code
status message)

that is, the server has found, and
is sending the requested object

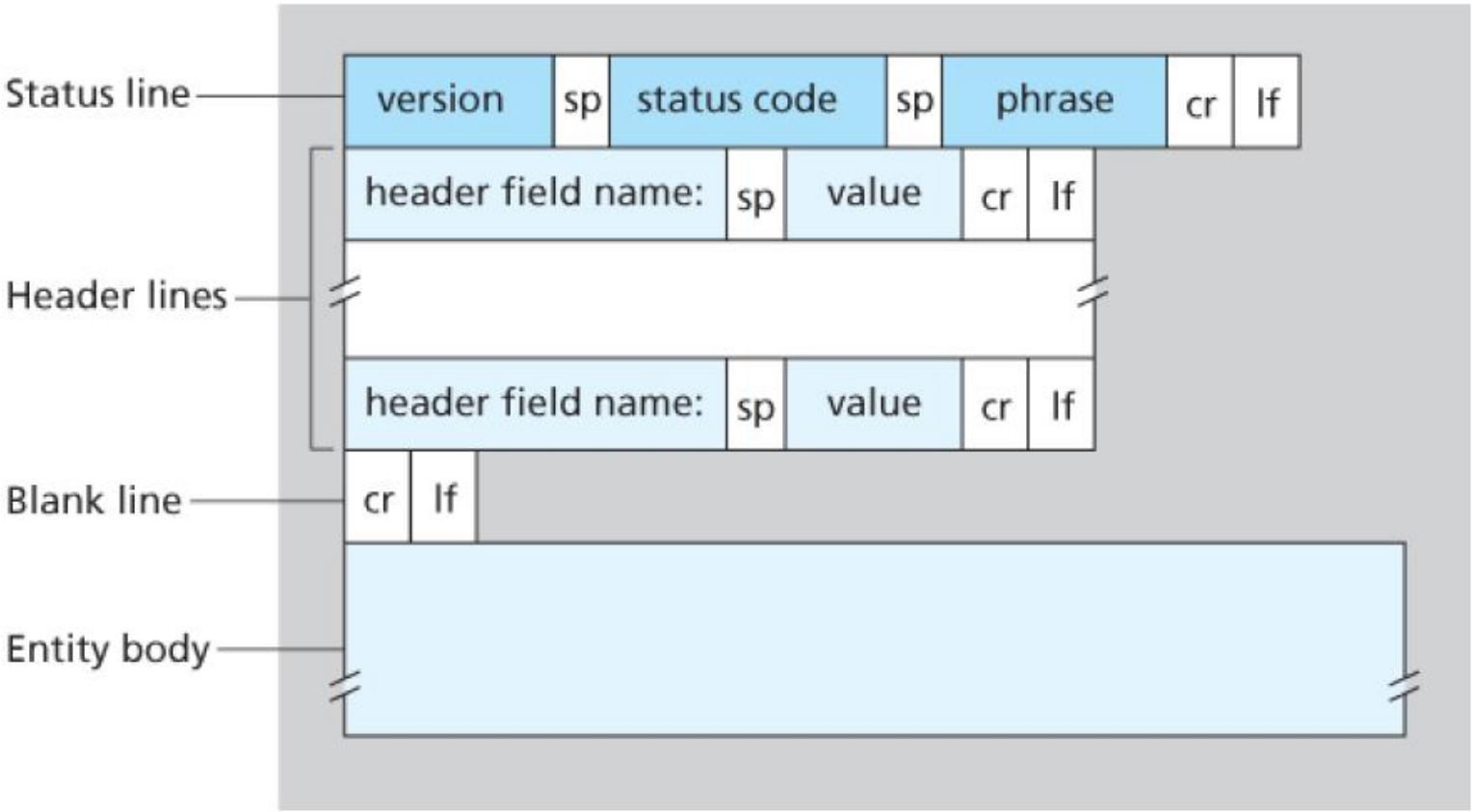
```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
   GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
   1\r\n
\r\n
data data data data data ...
```

header
lines

number of bytes

entity body,
e.g.,
requested
HTML file

HTTP response message



HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK

- request succeeded, requested object later in this msg

301 Moved Permanently

- requested object moved, new location specified later in this msg (Location:)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

telnet gaia.cs.umass.edu 80

- opens TCP connection to port 80 (default HTTP server port) at gaia.cs.umass. edu.
- anything typed in will be sent to port 80 at gaia.cs.umass.edu

2. type in a GET HTTP request:

GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu

- by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. look at response message sent by HTTP server!

(or use Wireshark to look at captured HTTP request/response)

User-server state: cookies

HTTP is Stateless, and servers handle thousands of simultaneous TCP connections.

However, it is often desirable for a Web server to identify users.

- **Web servers use cookies**

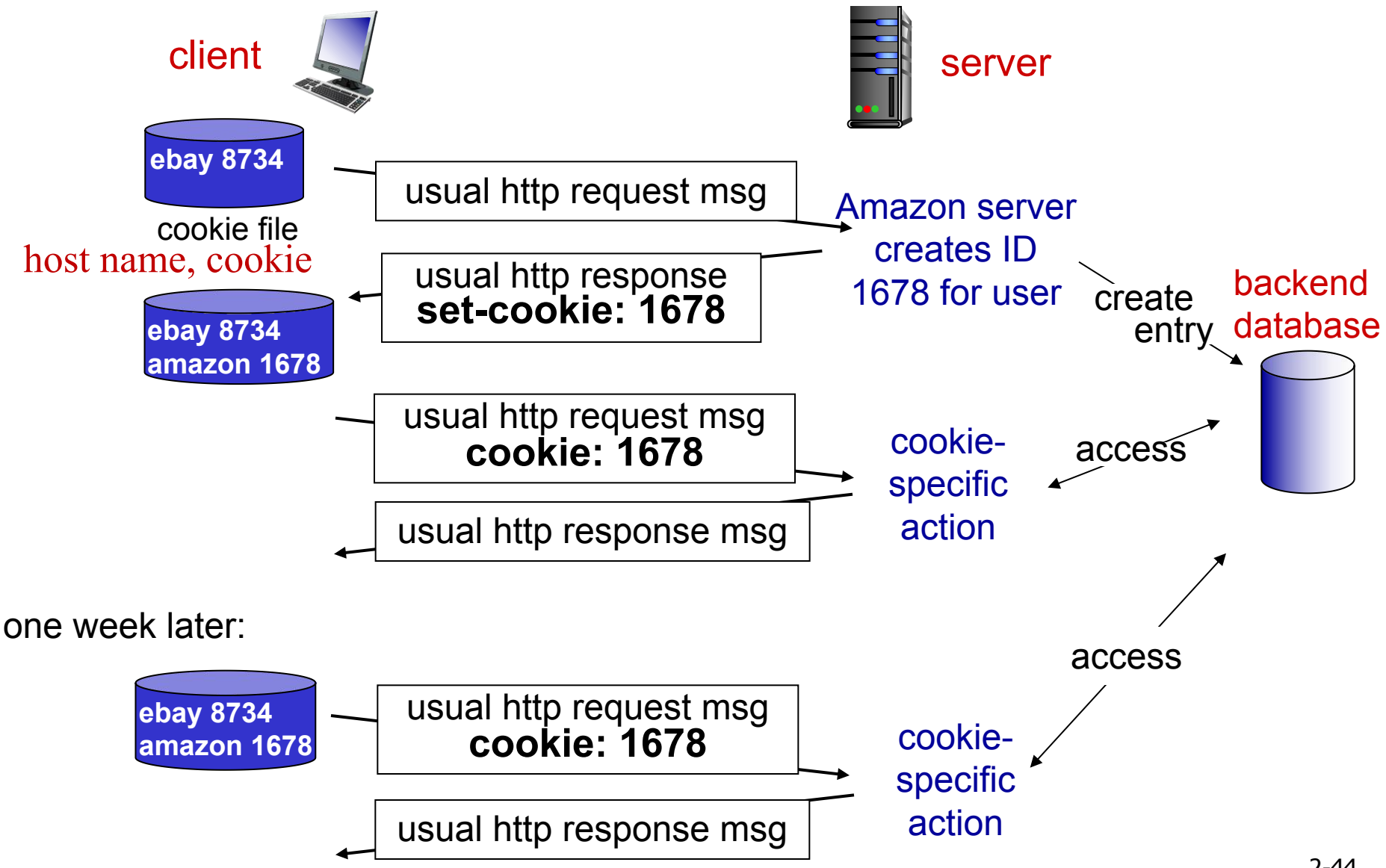
Four components:

1) cookie header line of HTTP *response* message
2) cookie header line in next HTTP *request* message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web server

example:

- Susan always access Internet from PC
- visits specific e-commerce site for first time
- when initial HTTP requests arrives at server, server creates:
  - unique ID
  - entry in backend database for ID

Cookies: keeping "state" (cont.)



Cookies (continued)

- Cookies are associated with web browser
- If Susan also registers herself with Amazon, the database can associate Susan's name with her identification number (cookies).

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session on top of stateless HTTP

aside

cookies and privacy:

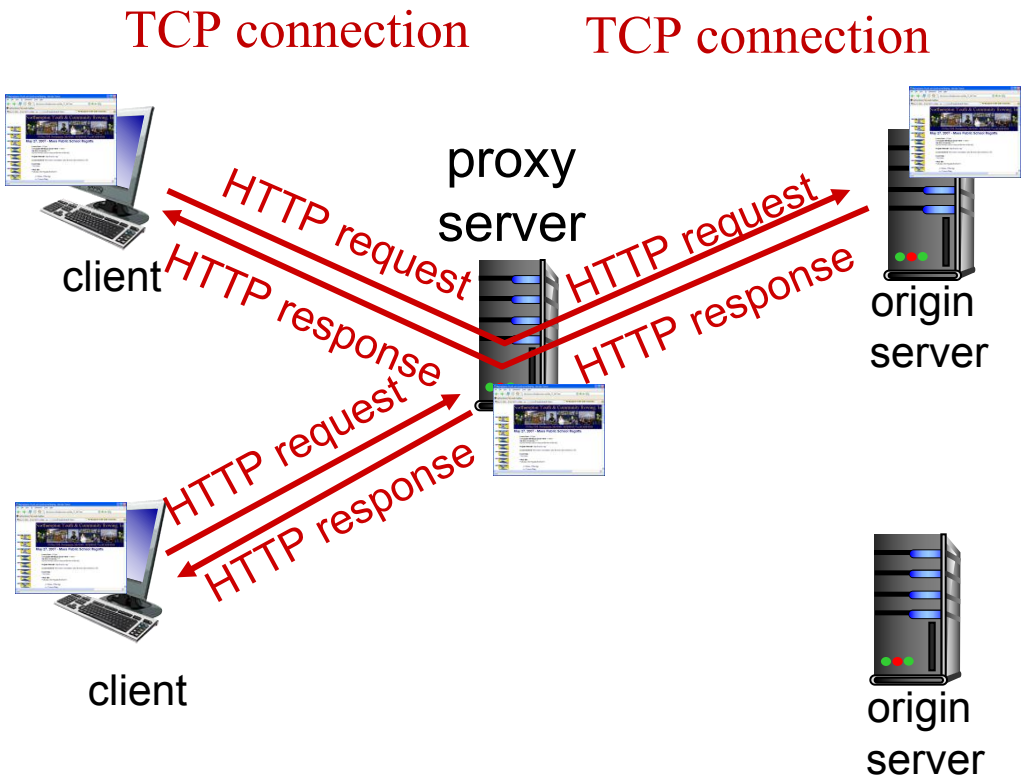
- cookies permit sites to learn a lot about you
- you may supply name and e-mail to sites

Web caches: proxy （代理） server

goal: satisfy client request without involving origin server

Browser sends all HTTP requests to cache

- object in cache: cache returns object
- else cache requests object from origin server, then returns object to client



More about Web caching

- Cache acts as both client and server
  - server for original requesting client
  - client to origin server
- typically cache is installed by ISP (university, company, residential ISP)

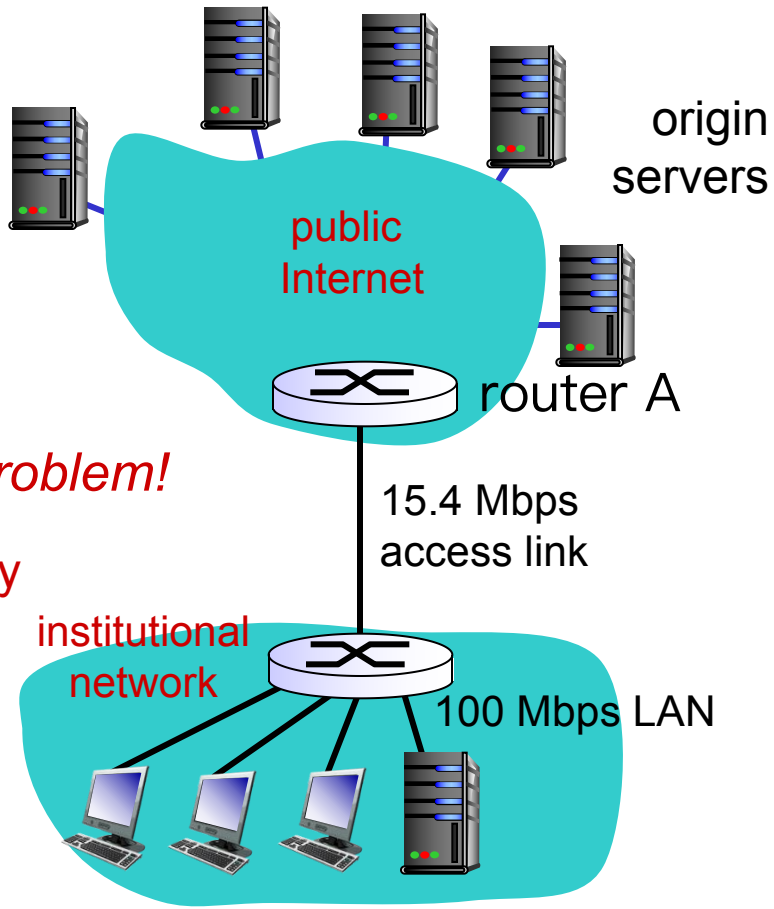
Why Web caching?

- reduce response time for client request (bottleneck bandwidth)
- reduce traffic on an institution's access link
- Internet dense with caches: enables "poor" content providers to effectively deliver content (so too does P2P file sharing)

Caching example:

Assumptions:

- avg object size: 1M bits
- avg request rate from browsers to origin servers:15 requests/sec
- avg data rate to all browsers: 15 Mbps
- RTT from router A to any origin server: 2 sec → "Internet delay"
- access link rate: 15.4 Mbps



problem!

Consequences:

Traffic intensity

- LAN utilization: 15M/100M=0.15
- access link utilization =15/15.4= 0.974
- total delay = Internet delay + access delay + LAN delay

  = 2 sec + minutes + milliseconds

Caching example: fatter access link

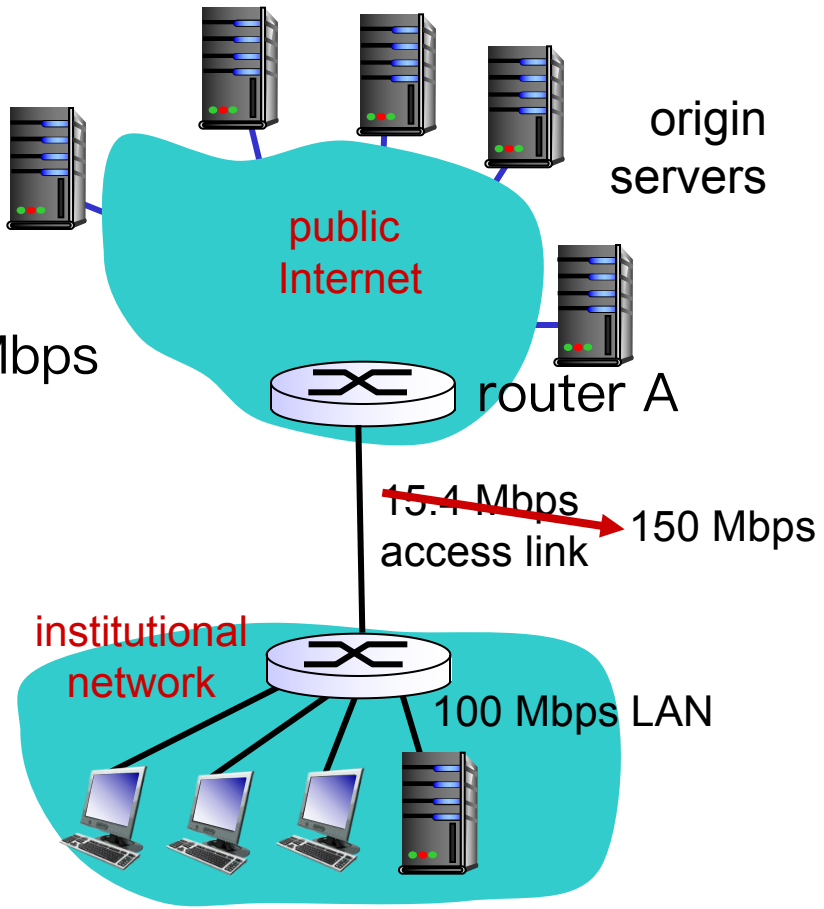
assumptions:

- avg object size: 1M bits
- avg request rate from browsers to origin servers:15/sec
- avg data rate to browsers: ~~15 Mbps~~ 150 Mbps
- RTT from router A to any origin server: 2 sec
- access link rate: 15.4 Mbps

consequences:

- LAN utilization: 0.15
- access link utilization = ~~0.974~~ 0.1
- total delay = Internet delay + access delay + LAN delay

  = 2 sec + ~~minutes~~ + milliseconds
  milliseconds



Cost: increased access link speed (not cheap!)

Caching example: install local cache

assumptions:

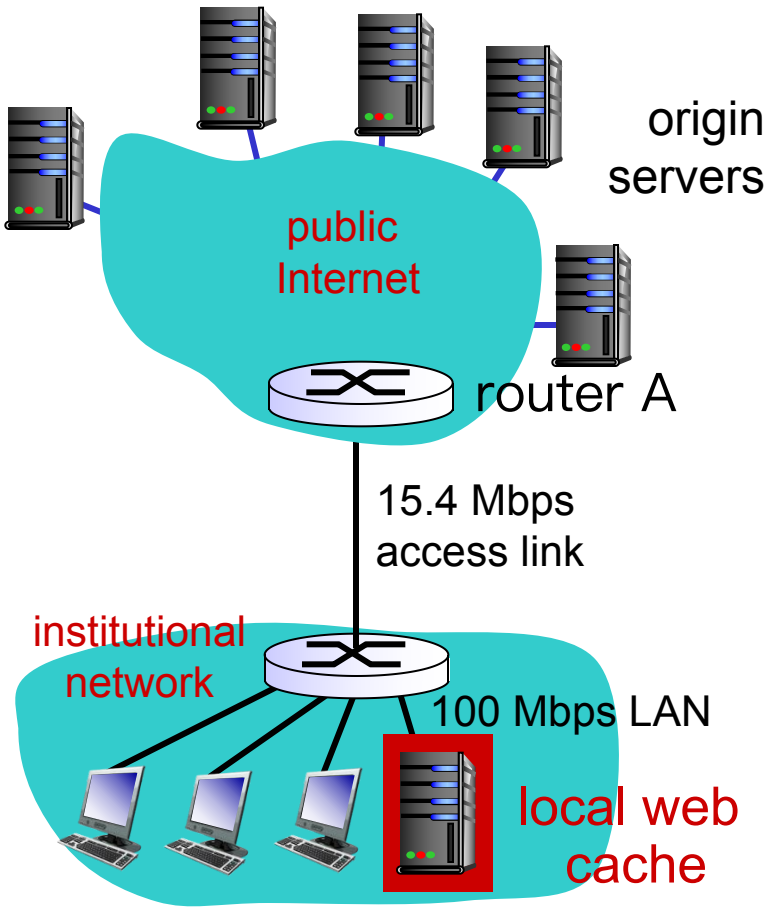
- avg object size: 1M bits
- avg request rate from browsers to origin servers:15/sec
- avg data rate to browsers: 15 Mbps
- RTT from router A to any origin server: 2 sec
- access link rate: 15.4 Mbps

consequences:

- LAN utilization: 0.15
- access link utilization = ?
- total delay = ?

How to compute link utilization, delay?

Cost: web cache (cheap!)



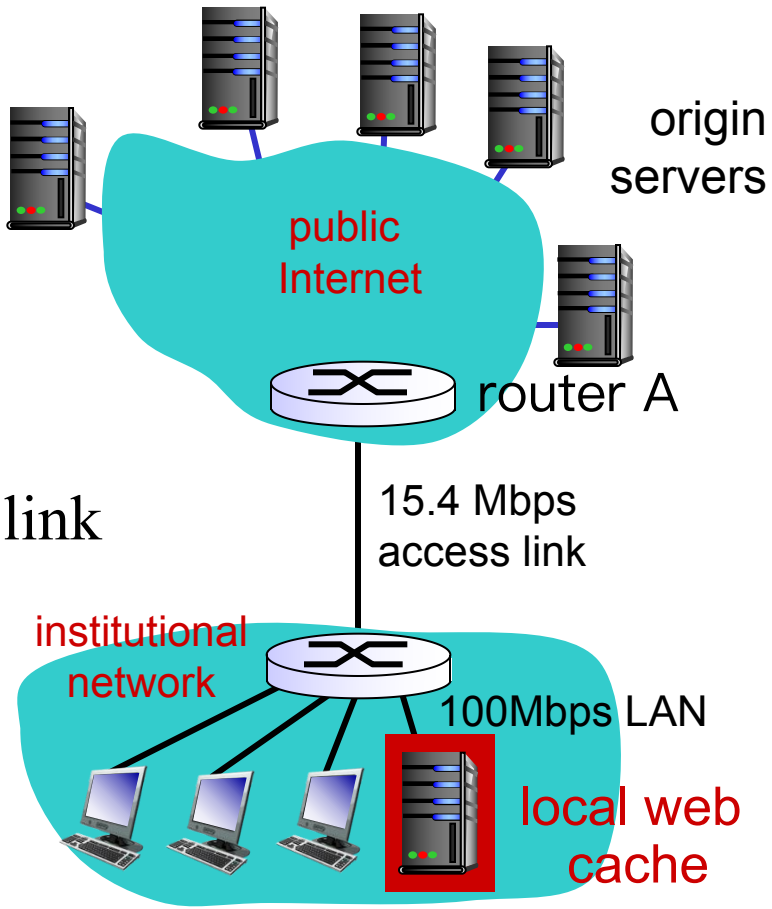
Hit rates: the fraction of requests that are satisfied by a cache. Typically, 0.2—0.7.

Caching example: install local cache

Calculating access link utilization, delay with cache:

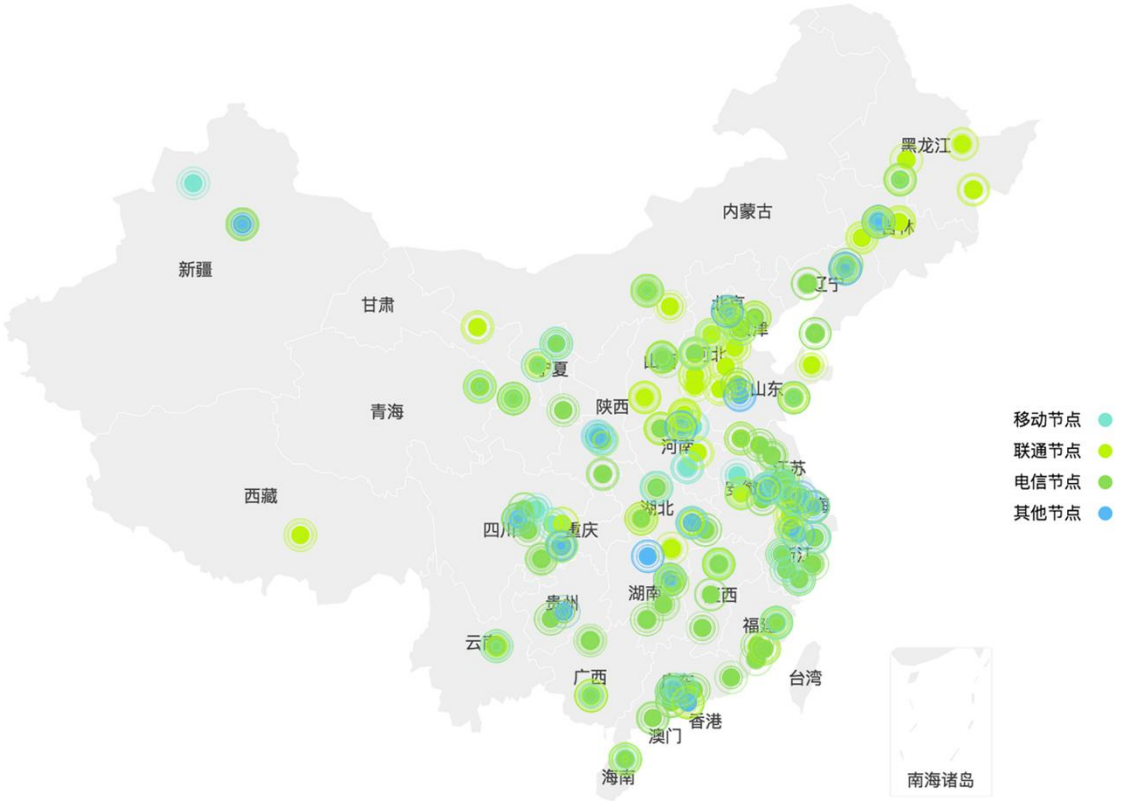
- suppose cache hit rate is 0.4
  - 40% requests satisfied at cache, 60% requests satisfied at origin
- access link utilization:
  - 60% of requests use access link
- data rate to browsers over access link

  = 0.6*15 Mbps = 9 Mbps
  - utilization = 9/15.4 = 0.58
- Average delay
  - = 0.6 * (delay from origin servers) +0.4 * (delay when satisfied at cache)
  - = 0.6 (2.01) + 0.4 (~msecs) = ~ 1.2 secs
  - less than with 150 Mbps link (and cheaper too!)



Typically, a traffic intensity less than 0.8 corresponds to a small delay, say, tens of milliseconds

Content Distributed Networks (CDN)



A CDN company installs many geographically distributed caches throughout the Internet, thereby localizing much of the traffic.

Proxy and Reverse Proxy

Conditional GET

The copy of an object residing in the cache may be out-of-date:

Conditional GET

- GET method
- If-Modified-Since

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24
```

Goal: allows a cache to verify that its objects are up to date

- don't send object if cache has up-to-date cached version
- no object transmission delay
- lower link utilization

Conditional GET

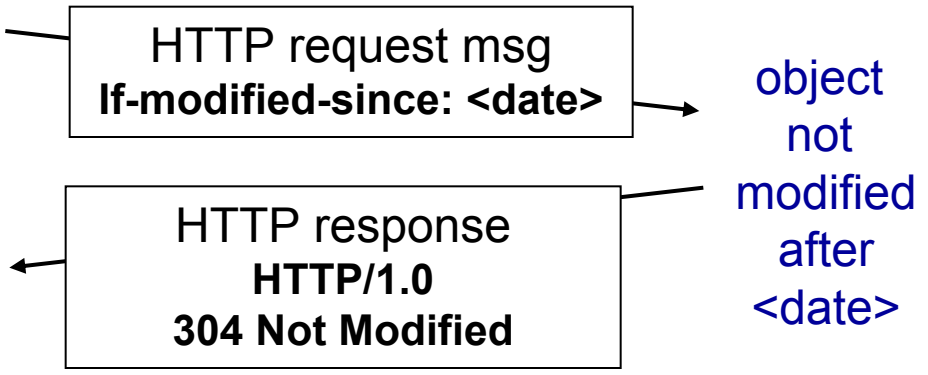
When a browser requests an object via proxy cache:

- *Proxy cache:* specify date of cached copy in HTTP request

  If-modified-since: <date>

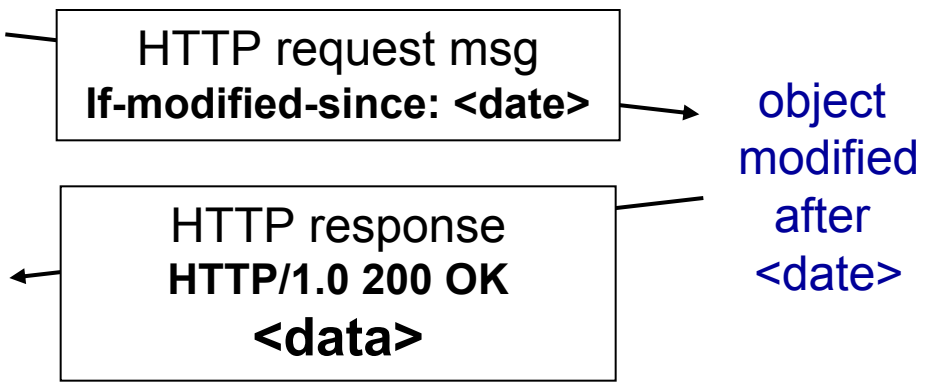
- *Server:* response contains no object if cached copy is up-to-date:

  HTTP/1.0 304 Not Modified




```
HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)

(empty entity body)
```



Chapter 2: outline

2.1 Principles of network applications

2.2 Web and HTTP

2.3 electronic mail

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P applications

2.6 video streaming and content distribution networks

2.7 socket programming with UDP and TCP